ihatepdf – Full Technical Architecture & Development Documentation

1. Project Overview

ihatepdf is a web-based document utility platform providing tools to manipulate PDF files.

The system allows users to merge, split, compress, convert, and edit PDF documents directly from the browser.

The platform is designed with a microservice-friendly architecture enabling scaling for millions of users.

2. Product Objectives

Provide a simple interface for document manipulation.

Support a wide range of file conversions and editing operations.

Ensure fast processing time with minimal latency.

Allow scalability to handle high traffic and heavy processing workloads.

3. Core Features

Merge multiple PDF files into a single document.

Split a PDF into multiple documents or extract page ranges.

Compress PDFs with configurable compression levels.

Convert between PDF and common document formats such as Word, Excel, PowerPoint, and images.

4. Editing Features

Rotate pages.

Add page numbers.

Insert watermarks.

Add digital signatures.

Basic annotation support.

5. Security Features

Password protect PDFs.

Unlock password protected PDFs.

Redact sensitive information.

Automatic deletion of uploaded files after processing.

6. File Format Support

Supported input formats: PDF, DOCX, XLSX, PPTX, JPG, PNG.

Supported output formats: PDF, DOCX, XLSX, PPTX, JPG.

7. High Level System Architecture

User Browser → CDN → Frontend Web App → API Gateway → Processing Backend → Worker Queue → File Storage → Result Delivery.

8. Frontend Architecture

The frontend is built using a component based framework.

It provides pages for each tool such as compress, merge, convert etc.

It communicates with the backend using REST APIs.

Frontend Technology Stack

Framework: Next.js

UI Library: Tailwind CSS

State Management: React Query

File Upload Handling: HTML5 drag and drop API

9. Backend Architecture

The backend exposes REST APIs for each PDF operation.

Each request triggers a processing job executed by a worker.

The system uses asynchronous processing to handle large workloads.

Backend Technology Stack

Backend Framework: FastAPI

Language: Python

API Layer: RESTful endpoints

10. Processing Engine

PDF processing uses specialized libraries for manipulation and conversion.

Large tasks are handled asynchronously by background workers.

Processing Libraries

PDF manipulation: PyPDF2

Text extraction: pdfplumber

Compression engine: Ghostscript

OCR engine: Tesseract

Document conversion: LibreOffice

11. Task Queue System

Background workers process heavy PDF operations.

Tasks are pushed into a queue and processed asynchronously.

Queue Technology

Queue broker: Redis

Worker framework: Celery

12. Storage Layer

Temporary files are stored during processing.

Files are automatically deleted after completion.

Storage Options

Development: local file storage.

Production: Amazon S3 object storage.

13. Database Design

A lightweight database is used to store user accounts, usage logs, and job metadata.

Table	Fields
Users	id, email, password, created_at
Jobs	id, user_id, job_type, status, created_at
Files	id, job_id, file_path, expiration_time

14. API Design

POST /merge

POST /split

POST /compress

POST /convert

GET /download/{file_id}

15. Deployment Architecture

The frontend is deployed on a CDN-backed hosting platform.

The backend runs on scalable containerized infrastructure.

Deployment Stack

Frontend hosting: Vercel

Backend hosting: Render or DigitalOcean

DNS and security: Cloudflare

16. Scalability Strategy

Use CDN caching for static assets.

Horizontal scaling of API servers.

Worker clusters for parallel processing.

Object storage for large files.

17. Monetization Model

Free tier with advertisements.

Premium subscription removing ads and increasing file limits.

18. Development Roadmap

Phase 1: MVP tools such as merge, split, compress.

Phase 2: add all conversion tools.

Phase 3: add OCR, authentication, and cloud integrations.

19. Security Considerations

Input validation for uploaded files.

Rate limiting API requests.

Encryption for stored files.

Automatic file deletion.

20. Conclusion

ihatepdf is designed to provide a scalable and efficient platform for document manipulation.

The architecture supports incremental expansion while maintaining performance and reliability.

Unit 5

By—

Y. Sathish Kumar

Assistant Professor, MVSREC

Generative Model:

- A generative model is a type of machine learning model that learns patterns from existing data in order to create new data.
- It focuses on understanding how data is generated rather than only predicting labels.
- After learning from training data, the model can produce new samples that resemble the original data.
- These samples are not copied from training data but are newly created based on learned patterns.
- Generative models are used to generate images, text, audio, and other forms of data.

How Generative Models Learn:

- Generative models learn by observing a large number of data examples during training.
- They identify common patterns such as shapes, structures, styles, or relationships present in the data.
- The model gradually learns the probability distribution that explains how the data is formed.
- After training, the model can sample from this learned distribution to create new data.
- The generated data appears realistic because it follows the same statistical patterns as the original dataset.
- This process enables the model to produce new, meaningful examples rather than copying existing ones

- When you teach a child to draw animals, the child does not memorize each picture exactly.
- Instead, the child observes many examples and slowly learns common patterns such as body shape, number of legs, eyes, ears, and tails.
- Over time, the child forms an internal understanding of what generally makes an animal look like an animal.
- Later, the child can draw a new animal that was never shown before by combining these learned features in a meaningful way.
- A generative model in deep learning works in the same manner.
- During training, the model is shown many data samples, such as images, text, or audio.

- Using neural networks, it learns the underlying structure and statistical patterns present in the data.
- Rather than storing the examples, the model learns a representation of the data distribution.
- After training, the model can generate new samples by using this learned distribution, producing data that looks realistic but is not an exact copy of the training input.
- This analogy helps explain why generative models are capable of creating new data.
- Just as a child uses what they have learned to create a new drawing, a deep generative model uses learned patterns and probabilities to generate new data that looks realistic and similar to the training data.

Types of Generative Models:

Probabilistic Models:

- Based on statistical distributions and dependencies.

Neural Network–Based Models:

- Deep learning models that learn complex data patterns.

Other Advanced Models:

- Modern architectures with exact likelihoods or energy functions.

Probabilistic Models:

Bayesian Networks

- Graph-based models showing dependencies between variables.
- Useful for causal reasoning, uncertainty modeling, and decision-making.

Hidden Markov Models (HMMs)

- Sequence models with hidden states and observable outputs.
- Widely used in speech recognition, time-series prediction, and NLP tasks.

Neural Network–Based Models:

Generative Adversarial Networks (GANs)

- Generator creates synthetic data.
- Discriminator classifies real vs. fake.
- Known for producing highly realistic images, videos, and audio.

Variational Autoencoders (VAEs)

- Encoder compresses input into latent space.
- Decoder reconstructs data from latent code.
- Can generate new, smooth variations of learned data.

Applications of Generative Models:

- Generative models are used across many domains to create new data, improve existing data, and support intelligent decision-making.
- **Key application areas include:**
- **Image Generation:** Creating realistic images, editing photos, restoring old images, and translating sketches to photos.
- **Text Generation:** Producing human-like text for chatbots, summaries, translations, and content creation.
- **Audio & Music Generation:** Generating natural speech for assistants and composing new music pieces.
- **Synthetic Data:** Building artificial datasets for privacy, training ML models, and data augmentation.
- **Healthcare & Drug Discovery:** Enhancing medical images and designing new molecules for faster research.

Challenges and Limitations of Generative Models:

- Generative models are powerful, but they come with several important challenges that affect their reliability and real-world use.
- **High Computational Requirements:** Training large models (especially GANs and diffusion models) demands powerful GPUs, long training times, and large memory resources.
- **Variable Output Quality:** Generated outputs can be incorrect, blurry, or unrealistic when data is limited, models are poorly trained, or tasks are too complex.
- **Security Risks:** These models can generate fake images, videos, or text, enabling misinformation, impersonation, and content manipulation.
- **Ethical and Trust Concerns:** The ability to create highly realistic synthetic content raises issues such as deepfakes, privacy violations, and potential misuse.
- **Data Dependence and Bias:** Output quality strongly depends on training data. If data is biased or incomplete, the model will reproduce those biases in its generation.

Discriminative model:

- A discriminative model is a type of machine learning or deep learning model that focuses on distinguishing between different classes or categories in a dataset.
- Its primary goal is to learn how to assign the correct label to an input based on observed features.
- Instead of modeling how the data is generated, the model directly learns the relationship between the input and the output.
- Discriminative models work by estimating the conditional probability of the output given the input.
- Through training on labeled data, the model learns decision boundaries that separate one class from another.

Cont..

- These boundaries help the model identify which class an input sample belongs to.
- Because of this approach, discriminative models are highly effective for tasks such as classification, detection, and recognition.
- In simple terms, a discriminative model learns what makes one class different from another.
- For example, in spam email detection, the model is trained on emails labeled as spam or non-spam.
- It learns patterns such as specific words, phrases, or structures that help distinguish spam from legitimate emails.
- When a new email arrives, the model uses the learned decision boundaries to classify it as spam or not spam.

How discriminative models work:

- Discriminative models work by learning the conditional probability $P(Y \mid X)$,which represents the probability of an output label Y given an input X.
- Their main objective is to correctly assign labels to inputs rather than to understand how the data itself is generated.
- Unlike generative models, discriminative models do not attempt to model the full data distribution of each class.
- Instead, they concentrate only on the differences between classes.
- First, the model processes the input data and extracts important features that are useful for distinguishing between categories.

- These features may be automatically learned by the model or provided as input, depending on the algorithm. .
- Next, during training, the model learns the relationship between these features and the corresponding class labels.
- By observing many labeled examples, it adjusts its parameters to form decision boundaries that separate one class from another.
- These boundaries represent regions in the feature space where data points are classified differently.
- Finally, during prediction, when a new input is given, the model analyzes its features and determines on which side of the learned decision boundary the input lies.
- Based on this position, the model assigns the most likely label to the input.
- This process makes discriminative models highly effective for tasks such as classification and recognition.

Examples of Discriminative Models:

- Several common machine learning models follow the discriminative approach.
- Logistic Regression is widely used for binary classification problems and learns a decision boundary that separates two classes based on input features.
- Support Vector Machines identify the optimal separating hyperplane by maximizing the margin between classes, which helps improve classification robustness.
- Neural Networks are trained discriminatively to map inputs directly to output labels by learning complex nonlinear relationships.
- Convolutional Neural Networks, a specialized type of neural network, are mainly used for image-related tasks and learn hierarchical visual features to accurately classify images.

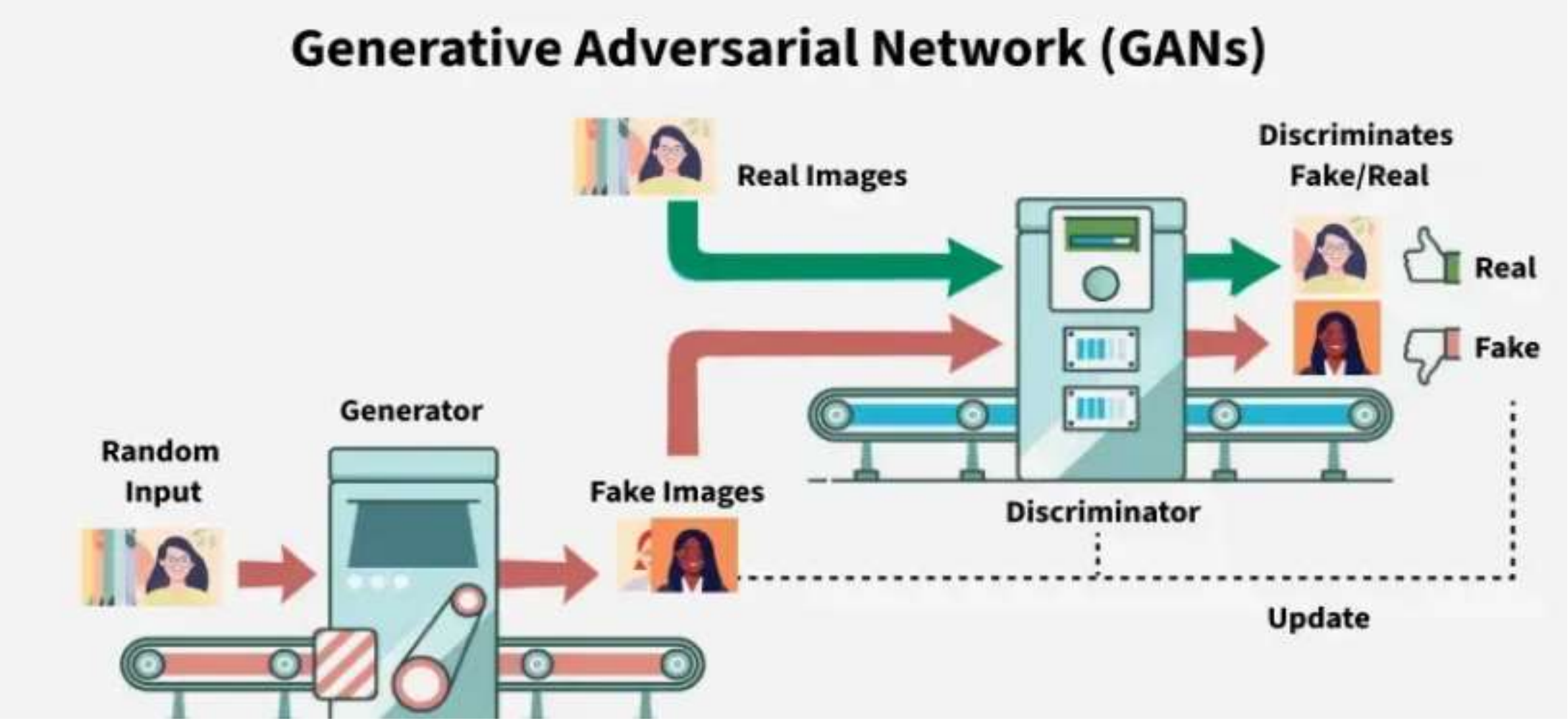
Advantages and Limitations of Discriminative Models:

- One major advantage of discriminative models is their efficiency, as they focus only on learning decision boundaries rather than modeling the full data distribution.
- They often achieve high accuracy in classification tasks, especially when trained on large amounts of labeled data.
- Discriminative models are also easier to train and optimize compared to many generative models.
- However, a key limitation is that they cannot generate new data since they do not model the underlying data distribution.
- Another limitation is their dependence on labeled datasets, which may be costly or difficult to obtain.
- As a result, their performance strongly depends on the quality and quantity of labeled training data.

Generative Adversarial Network (GAN):

- Generative Adversarial Networks (GANs) help machines to create new, realistic data by learning from existing examples.
- It is introduced by Ian Goodfellow and his team in 2014 and they have transformed how computers generate images, videos, music and more.
- Unlike traditional models that only recognize or classify data, they take a creative way by generating entirely new content that closely resembles real-world data.
- This ability helped various fields such as art, gaming, healthcare and data science

Architecture of GAN:



Architecture of GAN:

- GAN consist of two main models that work together to create realistic synthetic data.
- They are:
  1. Generator Model
  2. Discriminator Model

1. Generator Model:

- The generator is a deep neural network that takes random noise as input and generates data samples such as images, text, or audio.
- Its goal is to learn the underlying patterns of the real data so that the generated samples look realistic.
- During training, the generator updates its internal parameters using backpropagation based on feedback from the discriminator.
- The generator does not see real data directly; it learns indirectly by trying to fool the discriminator.
- As training progresses, the generator improves its ability to produce realistic samples.

Generator Loss Function:

- The objective of the generator is measured using the following loss function:

$$J_G = -\frac{1}{m}\sum_{i=1}^{m} \log D\left(G(z_i)\right)$$

- This loss function measures how good the generator is at fooling the discriminator.
- Here, m is the number of samples in a batch, and z_i represents random noise given as input to the generator.
- The term $G(z_i)$denotes the generated sample produced from the noise.
- The value $D\left(G(z_i)\right)$is the probability estimated by the discriminator that the generated sample is real.
- The generator aims to minimize this loss by increasing the value of $D\left(G(z_i)\right)$.
- In simple terms, the generator wants the discriminator to believe that the fake data it produces is real, meaning the predicted probability should be close to 1.
- When the generator successfully minimizes this loss, it produces more realistic samples that are harder for the discriminator to distinguish from real data.

Generator – Working:

- The generator is a neural network that creates new data (images, text, audio).
- It starts from random noise and gradually transforms it into meaningful, realistic data.
- During training, it learns how real data is structured through feedback.
- The following are the steps involved in generator:
  1. Input: Latent Vector (Random Noise)
  2. Fully Connected (Dense)
  3. LayersReshape / Projection to Feature Maps
  4. Upsampling / Transposed Convolution Layers
  5. Output Layer (Final Data Generation)
  6. Learning Through Feedback (Training Phase)

Step 1 – Input (Latent Vector / Noise):

- The generator begins with a random vector, called the latent vector (z).
- This vector is sampled from a simple distribution, usually Gaussian (Normal) or Uniform.
- Example:

  z = [0.2, -0.5, 1.1, ..., 0.7] (100 dimensions)

- Each value in z controls hidden properties of the output, such as:
  - shape
  - color
  - style
  - texture

Step 2 – Fully Connected (Dense) Layers:

- After the latent vector enters the generator, it first passes through several fully connected layers.
- In these layers, the values in the latent vector are combined and transformed through many connections.
- Every number in the latent vector influences many neurons in the next layer, allowing the generator to mix the information in different ways.
- As training progresses, the generator adjusts these connections so that certain combinations of numbers begin to produce useful internal patterns.
- Activation functions such as ReLU or Leaky ReLU are applied after each layer to introduce non-linearity.
- This helps the generator learn complex relationships rather than simple straight-line patterns.

Cont..

- Although this stage does not create any visible part of the final image, it plays an important role.
- These dense layers turn the simple random vector into a richer and more informative internal representation.
- This new representation carries early hints of structure that the generator will later expand into the full output.
- The output of this step is a higher-dimensional feature representation that contains more detailed and meaningful information than the original random vector, preparing it for the next stage where it will be reshaped into spatial form.

Step 3 – Reshape / Projection:

- After the fully connected layers produce a richer and higher-dimensional feature vector, the generator converts this long vector into a small spatial structure.
- This step creates the first form of an image-like layout.
- For example, if the dense layers output a large vector like 8192 values, it is reshaped into something like a $4 \times 4 \times 512$ feature map.
- This means the generator now has a small grid with many channels, where each channel holds different types of learned information.
- This spatial structure does not look like an image yet, but it introduces the idea of width, height, and depth.
- It gives the generator a starting layout that it can gradually expand into a full image in later steps.
- The main purpose of this step is to move from raw numerical features to an organized spatial format so that the generator can begin building shapes, edges, and patterns as it upsamples the feature map.

Step 4 – Upsampling / Transposed Convolutions:

- After the small feature map is created, the generator increases its size step by step until it becomes a full image.
- Each upsampling stage makes the image bigger and also adds more detail.
- The feature map grows through layers such as 4×4 to 8×8, then 8×8 to 16×16, 16×16 to 32×32, and finally 32×32 to 64×64.
- At each step, transposed convolutions or upsampling followed by convolution are used to enlarge the image.
- In the early expansions, the generator learns basic structure, such as the rough position of major shapes.
- In the middle layers, it starts forming clearer parts such as edges and noticeable patterns.
- In the final layers, it adds fine textures and small details that make the output look realistic.

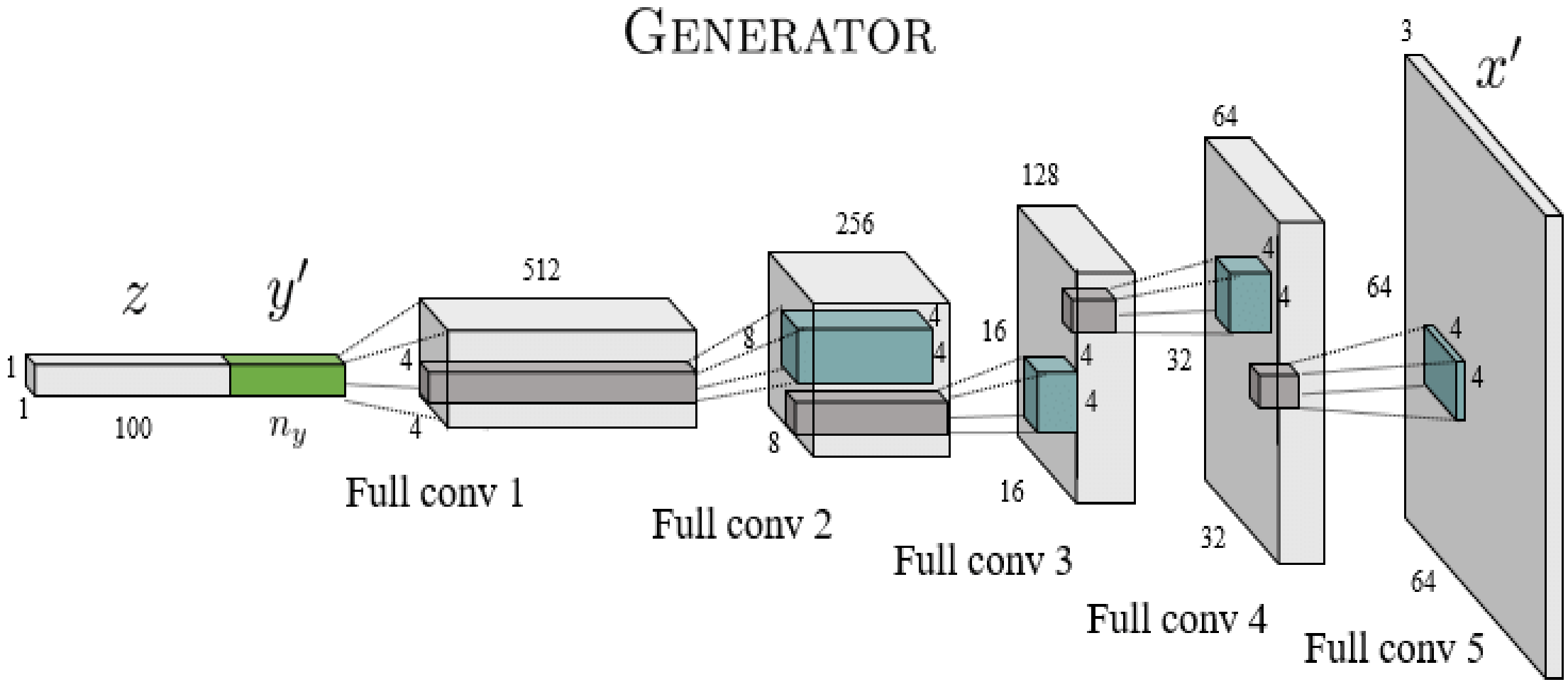
Step 5 – Output Layer:

- In the final stage, the generator converts the large feature map into the actual output format.
- For images, the last layer produces a $64 \times 64 \times 3$ structure where each value represents a pixel in the RGB image.
- For text, it produces a set of probabilities for each possible token, and for audio, it generates a waveform or spectrogram.
- This layer uses an activation function to make sure the values are in the correct range.
- Tanh is used for images so the pixel values fall between -1 and 1.
- Softmax is used for text so each set of values becomes a valid probability distribution.
- The output of this step is the final structured data that looks realistic because the earlier layers have already formed shapes, details, and patterns.

Step 6 – Learning During Training:

- At the beginning of training, the generator produces outputs that look completely random because it has not learned any patterns.
- As training continues, it improves by receiving feedback about how close its output is to real data.
- In GANs, the discriminator provides this feedback by telling the generator how “fake” its output looks.
- These errors are sent backward through the network using backpropagation.
- The generator updates its weights based on these errors, gradually learning which changes make the output look more realistic.
- Over time, the generator begins to produce outputs with correct structure, believable patterns, and natural variations that resemble the real data it was trained on.

Generator Architecture :



2. Discriminator Model:

- The discriminator acts as a binary classifier helps in distinguishing between real and generated data.
- It learns to improve its classification ability through training, refining its parameters to detect fake samples more accurately.
- When dealing with image data, the discriminator uses convolutional layers or other relevant architectures which help to extract features and enhance the model’s ability.

Discriminator Loss Function:

- The discriminator loss function measures how well the discriminator can distinguish between real data and fake data generated by the generator.
- The discriminator is trained to assign a high probability to real samples and a low probability to fake samples. To achieve this, its loss function is defined as:

$$J_D = -\frac{1}{m}\sum_{i=1}^{m} \log D\,(x_i) \;-\; \frac{1}{m}\sum_{i=1}^{m} \log(\,1 - D\big(G(z_i)\big))$$

- In this equation, m represents the number of samples in a batch.
- The term x_i denotes a real data sample taken from the training dataset, while $G(z_i)$represents a fake data sample generated by the generator from random noise z_i.

- The value $D(x_i)$is the probability predicted by the discriminator that the real sample $x_i$is actually real.
- Similarly, $D\big(G(z_i)\big)$is the probability predicted by the discriminator that a generated sample is real.
- For real data, the discriminator wants $D(x_i)$to be close to 1, which increases $\log D\ (x_i)$.
- For fake data, it wants $D\big(G(z_i)\big)$to be close to 0, which increases log(1−D(G(zi))). .
- By minimizing this loss function, the discriminator learns to correctly classify real samples as real and generated samples as fake.
- A lower discriminator loss means better classification performance.
- This loss function plays a crucial role in the adversarial training process of GANs.

Working of Discriminator:

- Its main role is to distinguish real data from fake data generated by the generator.
- By evaluating the authenticity of inputs, it provides feedback to the generator, helping it produce more realistic outputs over time.
- **Steps of Discriminator Working:**
  1. Input: Real and Generated Data
  2. Preprocessing / Normalization
  3. Convolutional / Feature Extraction Layers
  4. Downsampling / Pooling / Strided Convolutions
  5. Flatten / Fully Connected Layers
  6. Output Layer (Real vs Fake Score)
  7. Training with Labels (Loss Computation and Backpropagation)
  8. Interaction with Generator (Providing Feedback)

Step 1 – Input (Real and Fake Data):

- During training, the discriminator receives two types of input separately:
  - Real data from the training dataset, labelled as “real” (1)
  - Fake data generated by the generator, labelled as “fake” (0)
- For images, each input is a tensor of size 64×64×3 (height × width × channels).
- For text, inputs are sequences of token embeddings representing words or characters.
- The discriminator processes each input independently, evaluating whether it is real or fake.
- It does not compare real and fake images side by side; instead, it learns to detect patterns that distinguish real data from generated data across all inputs.

Step 2 – Preprocessing / Normalization:

- Before feeding data into the discriminator, inputs are preprocessed and normalized.
- For images, pixel values are typically scaled to a standard range (e.g., −1 to 1) or 0 to 1 to make learning more stable and faster.
- For text or other structured data, embeddings or token encodings convert the input into numerical vectors that the network can process.
- This step ensures that all inputs have a consistent format and scale, which helps the discriminator detect real vs fake patterns effectively.

Step 3 – Convolutional / Feature Extraction Layers:

- The discriminator uses convolutional layers (or other feature extraction layers for non-image data) to identify patterns in the input.
- Early layers detect low-level features such as edges, textures, and small shapes.
- Middle layers identify mid-level features like object parts, repeated patterns, and local structures.
- Deep layers capture global and abstract features, such as overall structure, symmetry, and realistic arrangement of elements.
- Through training, the network automatically learns which features distinguish real data from generated fake data, improving its ability to classify inputs correctly.

Step 4 – Downsampling / Pooling / Strided Convolutions:

- After extracting features, the discriminator gradually reduces the spatial dimensions of the input using pooling or strided convolutions.
- This step compresses information, allowing the network to focus on larger patterns and global structures.
- It helps the discriminator detect whether the overall layout, shapes, and arrangements are realistic.
- By combining information from different regions, the network can evaluate both local details and global consistency.
- Downsampling ensures that deeper layers can make decisions based on the entire input, not just small patches.

Step 5 – Flatten / Fully Connected Layers:

- After convolution and downsampling, the discriminator produces feature maps that contain learned patterns from the input.
- These feature maps have height × width × channels dimensions (for example, 8×8×256).
- To feed this into fully connected layers, the 3D feature map is flattened into a single 1D vector.
  - Example: 8×8×256 → 16,384-dimensional vector.
- This vector contains all the important information about the input, combining low-level, mid-level, and global features.
- Fully connected layers then process this vector to combine all features and prepare for the final decision (real or fake).

Step 6 – Output Layer (Real vs Fake Score):

- The final layer of the discriminator produces a single score or probability that indicates how likely the input is real.
- A sigmoid activation is typically used to map the output to a value between 0 and 1.
  - Close to 1 → the input is predicted as real
  - Close to 0 → the input is predicted as fake
- This score is then compared with the true label (1 for real images, 0 for generated images) to compute the discriminator loss.

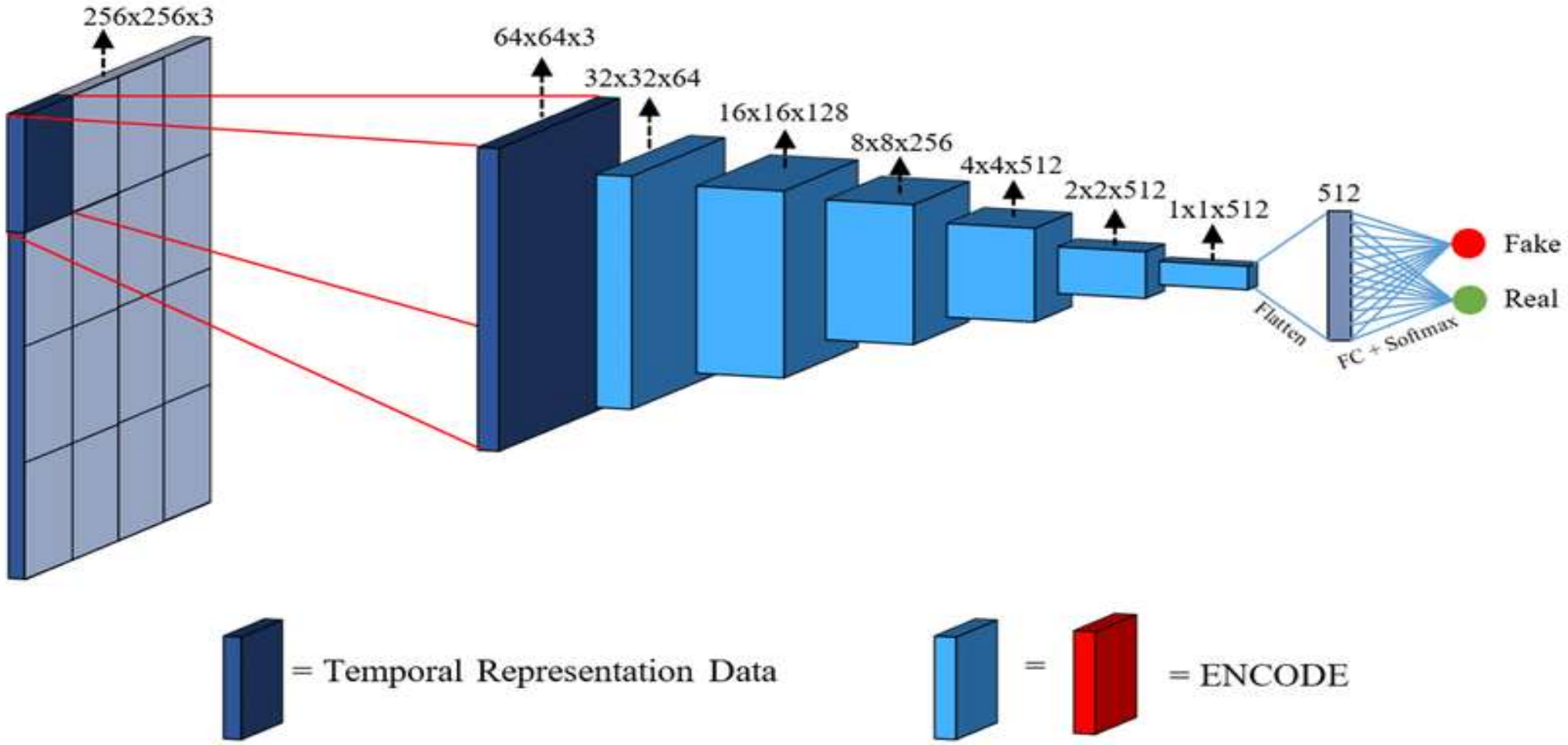
Step 7 – Training with Labels:

- During training, the discriminator uses labels to learn:
  - Real images are labelled 1
  - Fake images generated by the generator are labelled 0
- For each input, the discriminator predicts a score (real vs fake).
- The loss function measures how far the prediction is from the true label:
  - Predicting a real image as fake → increases loss
  - Predicting a fake image as real → increases loss
- Gradients of this loss are backpropagated through the network, updating all weights.
- Over time, the discriminator becomes more accurate at detecting subtle differences between real and generated data.

Step 8 – Interaction with Generator (Providing Feedback):

- The discriminator not only learns to detect real vs fake, but also guides the generator during training:
- The generator produces fake images to try and fool the discriminator.
- The discriminator evaluates these images and outputs a score indicating how real they look.
- This score is used to compute the generator's loss: the generator is rewarded when the discriminator is "fooled" and penalized when it is not.
- Backpropagation updates the generator's weights based on this feedback.
- Through this adversarial process:
  - The generator improves its ability to create realistic outputs.
  - The discriminator becomes better at spotting fake vs real.
- This competition drives both networks to improve simultaneously, leading to highly realistic generated data.

Discriminator Architecture:



MinMax Loss in Generative Adversarial Networks:

- In Generative Adversarial Networks, training is formulated as a MinMax optimization problem between two neural networks: the generator and the discriminator.
- The discriminator tries to maximize its ability to correctly classify real and fake data, while the generator tries to minimize this objective by fooling the discriminator.
- This opposing objective gives rise to the MinMax loss function:

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

- Here, G is the generator network and D is the discriminator network.

- The distribution $p_{data}(x)$ represents the true data distribution, and $p_z(z)$ represents the noise distribution used as input to the generator.
- The term $D(x)$ is the discriminator's estimate that a real sample is real, while $D\big(G(z)\big)$ is its estimate that a generated sample is real.
- The discriminator aims to maximize this objective by assigning high probabilities to real data and low probabilities to generated data.
- In contrast, the generator aims to minimize the same objective by making the discriminator believe that the generated data is real.
- This adversarial MinMax game continues until the generator learns to produce realistic samples and the discriminator can no longer reliably distinguish between real and fake data.

Summary-How Does a GAN Work:

- A Generative Adversarial Network works by training two neural networks together: the Generator and the Discriminator.
- These two networks compete with each other and improve over time through this competition.
- The generator tries to create realistic data samples, while the discriminator tries to correctly identify whether a given sample is real or generated.
- This adversarial process helps both networks learn effectively.

Role of the Generator:

- The generator starts with a random noise vector, which contains random numbers.
- Using this noise as input, the generator produces a fake data sample such as an image or text.
- Initially, these generated samples are poor and easily detected by the discriminator.
- Through training, the generator learns patterns from the feedback it receives and gradually produces more realistic outputs.
- Its main objective is to fool the discriminator into classifying fake data as real.

Role of the Discriminator:

- The discriminator receives both real data samples from the training dataset and fake data samples generated by the generator.
- It processes each input and outputs a probability value between 0 and 1 indicating how real the data is.
- A value close to 1 means the sample is likely real, while a value close to 0 means it is fake.
- The discriminator learns by comparing its predictions with the true labels.
- Over time, it becomes better at identifying fake samples.

Adversarial Learning:

- During training, both the generator and discriminator learn from each other’s mistakes.
- If the discriminator correctly identifies real data as real and fake data as fake, it improves its ability to classify data accurately.
- However, when the generator produces fake samples that the discriminator incorrectly classifies as real, the generator is rewarded for this success.
- At the same time, the discriminator is penalized for making an incorrect decision.
- This opposite learning objective creates an adversarial relationship between the two networks.

Generator’s Improvement:

- Every time the discriminator mistakes generated data as real, the generator learns what worked well.
- Using this feedback, the generator updates its parameters to produce even more realistic samples in the next iteration.
- Over many training cycles, the generator gradually learns the patterns and structure of the real data.
- As a result, the quality of the generated samples continuously improves.

Discriminator’s Adaptation:

- The discriminator also improves continuously throughout training.
- It updates its parameters whenever it incorrectly classifies fake data as real or real data as fake.
- This helps the discriminator learn more subtle differences between real and generated samples.
- The constant competition between the generator and discriminator makes both networks stronger and more accurate over time.

Training Progression:

- As training progresses, the generator continuously improves its ability to produce realistic data samples.
- At the same time, the discriminator becomes stronger but gradually finds it harder to distinguish between real and generated data.
- When the discriminator can no longer reliably tell real data from fake data, it indicates that the GAN has reached a well-trained or balanced state.
- At this stage, the generator is capable of producing high-quality and realistic synthetic data.
- These generated samples can then be used for various applications such as image generation, data augmentation

Types of GANs:

- There are several types of Generative Adversarial Networks (GANs), each designed to address specific challenges or improve performance in data generation tasks.
- Some commonly used GAN variants are:
  - Vanilla GAN
  - Conditional GAN (CGAN)
  - Deep Convolutional GAN (DCGAN)
  - Laplacian Pyramid GAN (LAPGAN)
  - Super-Resolution GAN (SRGAN)

Vanilla GAN:

- Vanilla GAN is the most basic and original form of a Generative Adversarial Network.
- It consists of two neural networks: a generator and a discriminator, both typically implemented using multi-layer perceptrons (MLPs).
- The generator creates fake data samples from random noise, while the discriminator tries to distinguish real data from fake data.
- Both networks are trained simultaneously using a minimax loss function.
- Although Vanilla GAN introduced the basic GAN concept, it often suffers from unstable training and mode collapse.

Conditional GAN (CGAN):

- Conditional GAN extends the Vanilla GAN by introducing an additional conditional input.
- This condition can be a class label or any extra information that guides the data generation process.
- Both the generator and the discriminator receive this conditional input along with their usual inputs.
- This allows the generator to produce specific types of data rather than random samples.
- For example, CGAN can generate images of a specific class such as handwritten digits or objects based on given labels.

Deep Convolutional GAN (DCGAN):

- Deep Convolutional GAN (DCGAN) is a popular and widely used type of GAN designed specifically for image generation tasks.
- Unlike Vanilla GANs, DCGANs use Convolutional Neural Networks (CNNs), which are very effective at learning spatial patterns such as edges, shapes, and textures in images.
- In DCGAN, the generator uses transposed convolution layers to gradually build an image from random noise, while the discriminator uses convolutional layers to extract features and classify images as real or fake.
- Traditional max-pooling layers are not used; instead, strided convolutions are applied to downsample feature maps, which helps maintain stable learning.
- Fully connected layers are removed, allowing the network to better preserve spatial relationships in images.
- Because of these architectural improvements, DCGANs produce higher-quality and more realistic images with more stable training compared to basic GANs.

Laplacian Pyramid GAN (LAPGAN):

- Laplacian Pyramid GAN generates images in a hierarchical, multi-resolution manner.
- It uses multiple generator–discriminator pairs corresponding to different image resolutions.
- Image generation starts at a low resolution and progressively adds finer details.
- Conditional GANs are often used at each level of the pyramid to refine the image.
- This approach produces highly detailed and realistic images while reducing noise.

Super-Resolution GAN (SRGAN):

- Super-Resolution GAN is designed to improve the resolution of low-quality images.
- The generator learns to map low-resolution images to high-resolution outputs.
- SRGAN uses adversarial loss along with perceptual loss to enhance visual quality.
- It produces sharper images with more realistic textures.
- SRGAN is widely used in applications such as medical imaging, satellite imaging, and photo enhancement.

Loss Functions in GANs:

- Loss functions guide how both the Generator (G) and Discriminator (D) learn during training.
- They measure how well each network is performing and provide gradients for weight updates.
- The discriminator’s loss measures how accurately it separates real images from fake images.
- The generator’s loss measures how well it fools the discriminator.
- GAN training is a minimax game, where:
  - The discriminator tries to maximize correct classification.
  - The generator tries to minimize the chance of being detected.
- The balance between both losses determines whether training is stable or collapses.

Discriminator Loss (Binary Cross-Entropy Loss):

- The discriminator gets two types of inputs during training:
  - Real data → label 1
  - Fake data from generator → label 0
- The discriminator's objective:
  - Predict **1** for real samples
  - Predict **0** for fake samples
- Loss formula (BCE):

$$LD = -[\log(D(real)) + \log(1 - D(fake))]$$

  - If D correctly identifies real as real → first term is small.
  - If D correctly identifies fake as fake → second term is small.
  - If D gets confused → loss increases.
- This loss ensures the discriminator becomes better at spotting fake images.

Generator Loss (Non-Saturating BCE Loss):

- The generator never directly compares its output with real images.
- It learns only through the discriminator's feedback.
- Goal of generator:
  - Make fake images look real enough that D(fake) → 1
- Loss formula:

$$LG = -\log(D(fake))$$

- If the discriminator thinks the fake image is real → low loss.
- If the discriminator easily rejects it → high loss.
- This loss pushes the generator to learn meaningful patterns and improve realism.

Original GAN Loss Is Unstable:

- The standard GAN loss faces several issues:
- **Vanishing gradients**:
  If the discriminator becomes too strong early, the generator receives almost no gradient.
- **Non-convergence**:
  Training can oscillate, with no stable improvement.
- **Mode collapse**:
  The generator may produce only one or few types of images.
- **Sensitive to hyperparameters** such as learning rate, batch size, and architecture.
- These limitations led to improved loss functions used in modern GANs.

Least Squares GAN (LSGAN) Loss:

- LSGAN replaces BCE with Mean Squared Error, which smooths gradients and stabilizes training.
- Discriminator loss:

$$LD = (D(real) - 1)^2 + (D(fake))^2$$

- Generator loss:

$$LG = (D(fake) - 1)^2$$

- Reduces the problem of vanishing gradients.
- Makes fake and real classes easier to separate.
- Produces more stable training and better image quality.
- Useful in tasks like image-to-image translation.

Wasserstein GAN (WGAN) Loss:

- WGAN uses a critic that gives real-valued scores instead of probabilities.
- Critic tries to give higher scores to real images and lower scores to fake images.
- Loss: LD = D(real) − D(fake) and LG = −D(fake).
- Scores change smoothly, so generator always receives useful gradients.
- Training becomes stable and avoids vanishing gradients.
- Loss values directly reflect image quality.
- Mode collapse reduces because critic penalizes lack of diversity.
- Requires weight clipping or gradient penalty for stable critic behavior.

Hinge Loss (Used in StyleGAN & BigGAN):

- Hinge loss trains the discriminator only on samples near the decision boundary.
- Loss: $L_D = \max(0, 1 - D(\text{real})) + \max(0, 1 + D(\text{fake}))$ and $L_G = -D(\text{fake})$.
- Real images should have $D(\text{real}) \geq 1$; fake images should have $D(\text{fake}) \leq -1$.
- Discriminator ignores already confident predictions, preventing overfitting.
- Generator receives stronger and more stable gradients.
- Improves sharpness and overall visual quality of generated images.
- More stable than original GAN loss and widely used in modern high-resolution GANs.
- Reduces training instability and leads to smoother convergence.

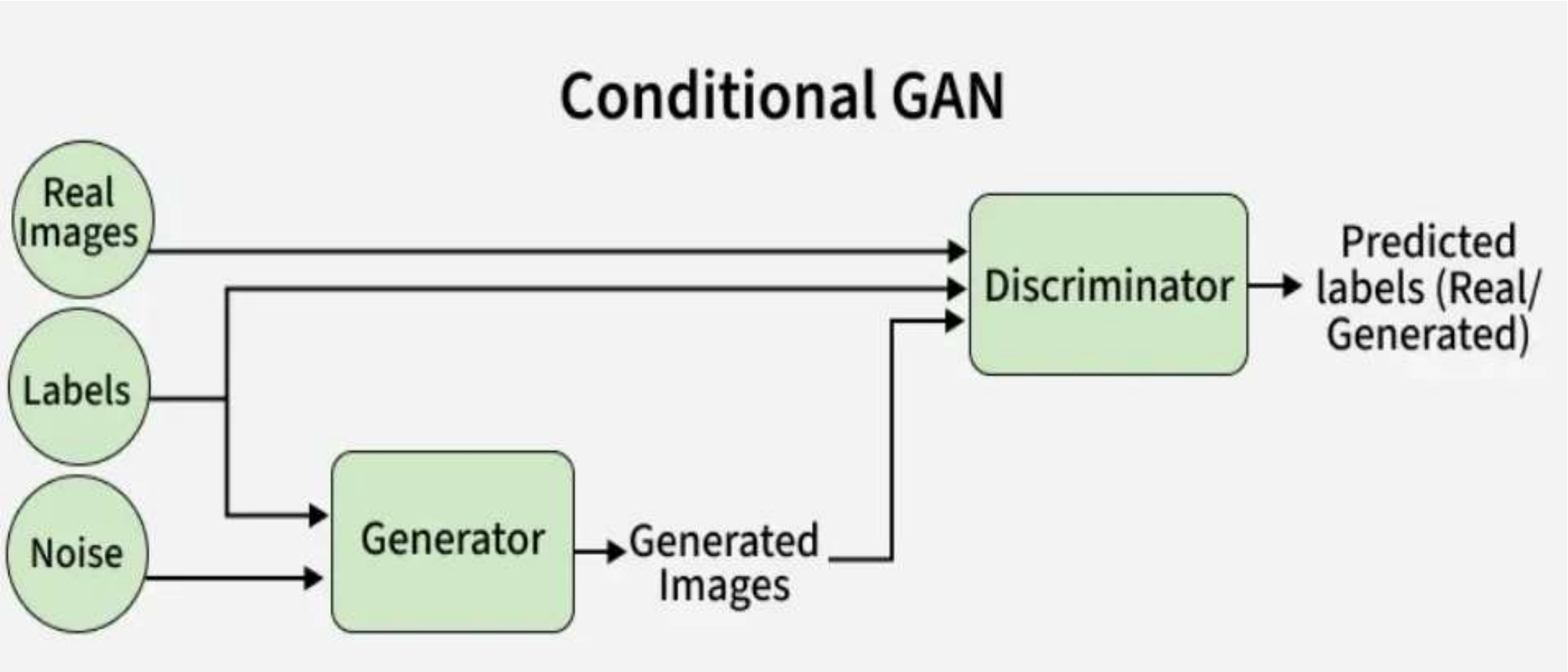
Conditional Generative Adversarial Network (CGAN):

- A Conditional Generative Adversarial Network (CGAN) is an advanced version of a Generative Adversarial Network (GAN) that can generate data based on a given condition such as a class label, category, or any additional information.
- In a standard GAN, the generator produces random samples without control over the output type.
- In contrast, a CGAN allows controlled generation, meaning we can decide what kind of output we want.

Example: MNIST Handwritten Digits:

- Consider the MNIST dataset, which contains handwritten digits from 0 to 9.
- A normal GAN may generate any random digit.
- A CGAN can be instructed to generate a specific digit, such as only digit “7”, by providing the label 7 as the condition.
- This makes CGANs very useful when we need class-specific or targeted data generation.

Architecture of Conditional GAN:



Architecture of Conditional GAN:

- A CGAN consists of two neural networks:
  1. Generator (G)
  2. Discriminator (D)
- The key idea is that both networks receive additional conditional information (y).

1. Generator in Conditional GAN (CGAN):

- The generator in a Conditional GAN (CGAN) is a neural network whose main task is to create fake data that looks similar to real data, while also matching a given condition such as a class label.

Inputs to the Generator:

- The generator takes two inputs:

  a) Random Noise (z)

  b) Condition (y)

a) Random Noise (z):

- z is a vector of random numbers (usually sampled from a normal or uniform distribution)
- Example: z = [0.23, -1.45, 0.78, …]
- Noise ensures:
  - Diversity in generated images
  - Different outputs even for the same condition
- Without noise, the generator would produce the same image every time

b) Condition (y):

- y represents additional information which is usually a class label
- Example (MNIST):

  $y = 0, 1, 2, \ldots, 9$

- The condition tells the generator what type of image to generate

Internal Working of the Generator:

Step 1: Input Preparation:

1. Noise Vector Creation:

- A random noise vector z is generated
- Example: 100-dimensional vector

2 Condition Encoding:

- The condition y (digit label) is converted into a numerical form
- Usually done using one-hot encoding
- Example for digit 5:
- y = [0 0 0 0 0 1 0 0 0 0]

Step 2: Combining Noise and Condition:

- The noise vector z and condition vector y are combined
- Common methods:
  - Concatenation
  - Embedding + concatenation

Example:

- Input to Generator = [z || y]
- This combined vector now contains:
  - Randomness (from z)
  - Control information (from y)

Step 3: Passing Through Fully Connected (Dense) Layers:

- After combining the noise vector and the condition label, the resulting input is passed into a single dense (fully connected) layer of the generator.
- This dense layer multiplies the input with learned weights and converts it into a higher-dimensional feature vector.
- An activation function such as ReLU or LeakyReLU is then applied to introduce non-linearity and highlight important features.
- The output of this single layer already contains condition-specific information learned from the training data.
- This feature vector is then reshaped and further processed to generate the final fake image.

Step 4: Feature Expansion (Image Formation):

- After the dense layer, the feature vector is reshaped into a small spatial form that can be converted into an image.
- This reshaped feature map is then passed through deconvolution (transposed convolution) layers in the generator.
- Each deconvolution layer increases the spatial dimensions of the feature map, gradually expanding it toward the final image size.
- During this expansion, the network adds visual structure such as strokes and basic shapes.
- As the data passes through successive layers, edges and finer details are refined.
- Finally, the generator produces a complete image of size 28×28 that looks like a realistic handwritten digit.

Step 5: Final Output Formation:

- During the generation process, the conditional information influences the network at every stage, guiding the shape, structure, and overall appearance of the image.
- Because the condition is embedded into the generator, features related to the given label are strengthened while irrelevant features are suppressed.
- For example, when the condition is $y = 5$, the generator emphasizes curved strokes and patterns typical of the digit 5 and avoids features of other digits.
- In the final stage, the processed feature map is passed through the output layer.
- This layer uses an activation function such as tanh or sigmoid to scale the output values into a valid pixel range.
- The final result is a 28×28 fake image that closely resembles a handwritten digit and correctly matches the given condition.

What the Generator Learns Over Time:

- During the initial stages of training, the generator produces blurry or meaningless images because it has not yet learned useful patterns.
- As training progresses, the generator starts forming rough shapes that resemble handwritten digits.
- In the later stages, the generated images become clear and realistic, closely matching real handwritten digits.
- This improvement happens because the generator continuously learns from the feedback provided by the discriminator.
- Whenever the discriminator correctly identifies a fake image, the generator adjusts its weights to reduce errors.
- Over time, this process helps the generator produce high-quality images that can successfully fool the discriminator.

2. Discriminator in Conditional GAN (CGAN):

- The discriminator in a Conditional GAN acts as a binary classifier whose main task is to decide whether the given input image is real or fake while also considering the provided condition.
- It receives two inputs: the data sample and the conditional information.
- The data sample can either be a real image taken from the training dataset or a fake image generated by the generator.
- Along with the image, the discriminator also receives the same condition used during generation, such as a class label.

Cont..

- During its operation, the discriminator processes the image together with the condition to verify two things simultaneously.
- First, it checks whether the image looks realistic enough to belong to the real dataset.
- Second, it checks whether the image actually matches the given condition.
- For example, in the MNIST dataset, if an image is provided with the label 3, the discriminator examines whether the image resembles a real handwritten digit and whether it truly looks like the digit 3.
- Based on this evaluation, the discriminator outputs a probability indicating whether the input is real or fake.

Internal Step-by-Step Working of the Discriminator in CGAN:

- The working of the discriminator begins when it receives two inputs: an image and its corresponding condition label.
- The image can be either a real sample from the dataset or a fake sample generated by the generator.
- At the same time, the condition label is converted into a numerical form, usually through one-hot encoding, so that it can be processed by the network.
- The condition information is then combined with the image data, either by concatenating it with the input or embedding it into the network layers.

Cont..

- Once the combined input is prepared, it is passed through convolutional layers that extract important visual features such as edges and shapes from the image.
- These layers help the discriminator understand the structure of the image while keeping the condition in mind.
- Activation functions like Leaky ReLU are applied after each layer to introduce non-linearity and ensure stable learning.
- As the data flows through deeper layers, the discriminator learns higher-level features that help distinguish real images from fake ones.

Cont..

- After feature extraction, the output is flattened into a single vector and passed through a dense layer.
- This dense layer combines all learned features and evaluates how realistic the image is and whether it matches the given condition.
- Finally, a sigmoid activation function is applied in the output layer to produce a probability value between 0 and 1.
- A value close to 1 indicates that the image is real and correctly conditioned, while a value close to 0 indicates that the image is fake or does not match the condition.

Adversarial Training Process:

- The generator and discriminator are trained simultaneously in a competitive manner:

Generator's Objective:

- Generate images that look real
- Ensure the images match the given condition
- Fool the discriminator into classifying fake images as real

Discriminator's Objective:

- Correctly identify real vs. fake images
- Check whether the image matches the condition
- This competition continues until:
  - The generator produces realistic, condition-based images
  - The discriminator can no longer easily distinguish fake images

Loss Function and Training in Conditional GAN (CGAN):

- The training of a Conditional GAN is guided by the min max loss function that creates a competition between the generator and the discriminator.

$$\min_{G} \max_{D} V\,(D, G) = E_{x \sim p_{data}(x)}[\log D\,(\,x \mid y\,)] + E_{z \sim p_z(z)}\left[\log\left(1 - D\big(G(\,z \mid y\,)\big)\right)\right]$$

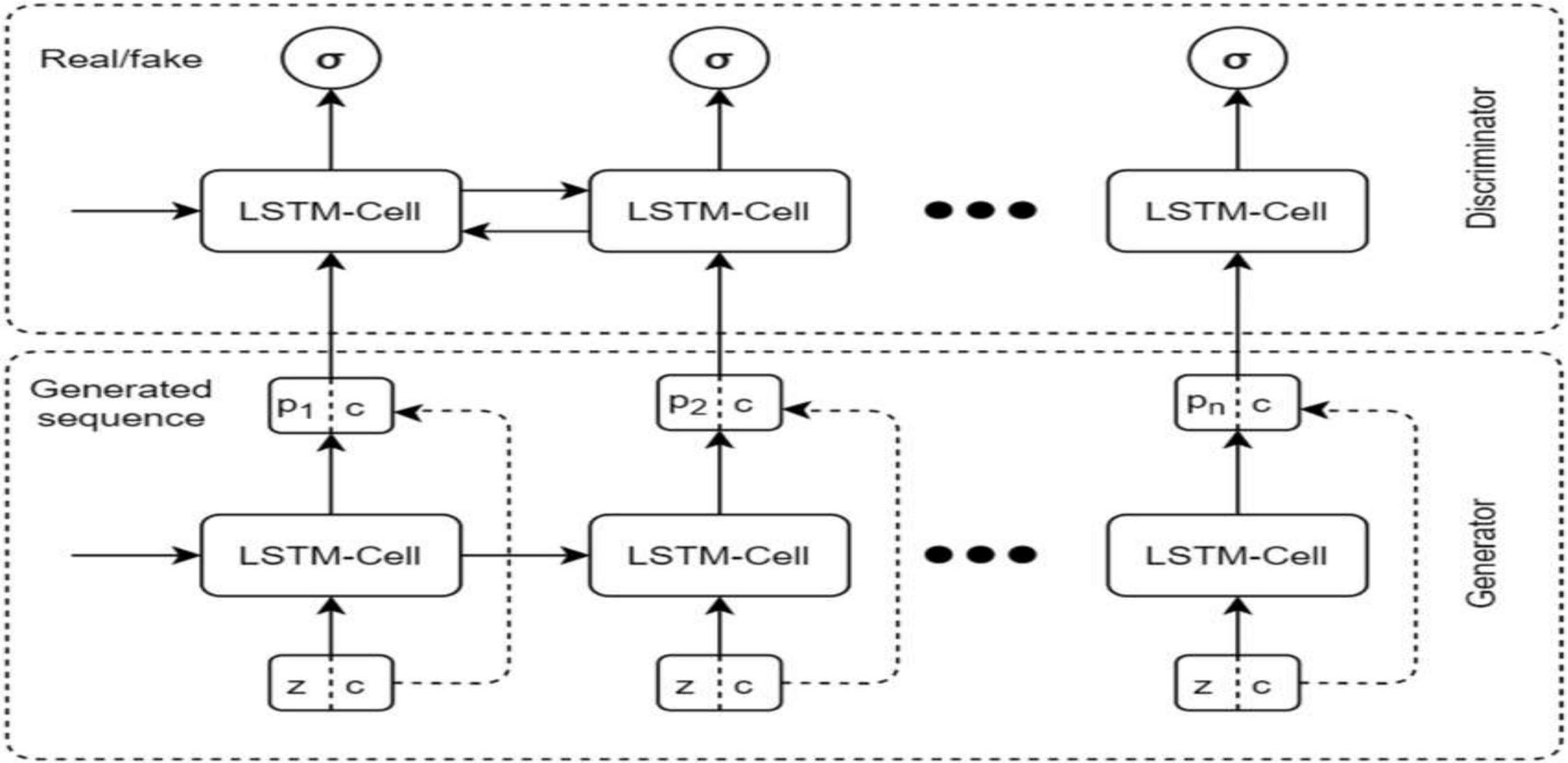
- This loss function is designed so that the generator tries to produce realistic data, while the discriminator tries to correctly identify real and fake data using the given condition.
- The first part of the loss function focuses on real data samples.
- It encourages the discriminator to assign a high probability to real images when they are given with the correct condition.
- For example, if a real handwritten digit image is labeled as “3,” the discriminator is rewarded for correctly recognizing it as a real digit 3.

- The second part of the loss function focuses on fake data generated by the generator.
- It pushes the generator to create images that the discriminator believes are real for the given condition.
- If the generator produces a fake image of digit “5,” the loss decreases only when the discriminator is fooled into thinking that the image is a real digit 5.
- Here, the expected value represents the average performance over many samples.
- The real data distribution refers to actual training data, while the noise distribution refers to random input given to the generator.
- During training, the discriminator improves its ability to distinguish real and fake images, and the generator improves its ability to fool the discriminator.
- This adversarial learning process continues until the generator produces highly realistic images that correctly match the given condition.

Recurrent Conditional GAN (RC-GAN):

- A Recurrent Conditional GAN (RC-GAN) is an extension of the Conditional GAN designed to generate sequence-based data like text, speech, or video.
- Unlike standard CGANs, which generate single images or data points, RC-GANs produce entire sequences, taking into account the temporal or sequential relationship between elements while conditioning on extra information such as labels or context.

Architecture:



Architecture:

- RC-GAN has two main parts: generator and discriminator, both using recurrent layers (like LSTM or GRU) to process sequences.
- The **generator** takes:
  - **Random Noise (z):** A vector of random values that introduces diversity.
  - **Condition (y):** Label or extra information that guides the output (e.g., sentiment, action type).
- The **discriminator** takes:
  - **Sequence data (x):** Real sequences from the dataset or fake sequences from the generator.
  - **Condition (y):** The same label or context to verify consistency.

Generator Working:

- The noise vector z and condition y are combined and embedded into a dense vector to initialize the recurrent network.
- At each time step, the recurrent unit (LSTM/GRU) generates one element of the sequence (e.g., one word in a sentence, one frame in a video) while considering its previous hidden state.
- The hidden state carries temporal context, ensuring that the sequence is coherent over time.
- Each output is passed through an activation layer or dense layer to map it to the desired data format.
- This process continues until the full sequence is generated, reflecting the condition.
- Example:
  For text generation with condition “positive sentiment,” the generator produces words one by one so that the whole sentence maintains positive sentiment.

Discriminator Working:

- The input sequence and condition are first embedded and combined.
- The recurrent layers process the sequence step by step, producing hidden states that summarize temporal patterns.
- At each step or at the end of the sequence, the network evaluates both realism and condition consistency.
- The final hidden state is passed through a dense layer with a sigmoid activation to output a probability between 0 (fake) and 1 (real).

Loss Function and Training:

- RC-GAN uses the same adversarial loss as CGAN:

$$\min_{G} \max_{D} V(D, G)$$
$$= E_{x \sim p_{data}(x)}[\log D(x \mid y)] + E_{z \sim p_z(z)}\left[\log\left(1 - D\left(G(z \mid y)\right)\right)\right]$$

- The discriminator tries to correctly classify real sequences as real and fake sequences as fake.
- The generator tries to fool the discriminator by producing sequences that are both realistic and consistent with the condition.
- Training alternates between updating the discriminator and generator until the generator produces high-quality sequences.

Applications of RC-GAN:

- Text generation: Sentiment-controlled sentences or stories.
- Speech synthesis: Generating audio conditioned on speaker identity or emotion.
- Video generation: Generating frame sequences based on actions.
- Time-series prediction: Creating future sequences conditioned on previous data.

Introduction to Pix2Pix:

- Pix2Pix is a supervised image-to-image translation technique based on Conditional Generative Adversarial Networks (CGANs).
- It was introduced by researchers at UC Berkeley in 2017 to learn a mapping between paired images.
- The main objective of Pix2Pix is to translate an input image from one domain into a corresponding output image in another domain.
- Unlike traditional image processing methods, Pix2Pix learns this translation automatically from data.
- It is widely used in applications such as sketch-to-photo conversion, map-to-satellite image translation, and image colorization.

Image-to-Image Translation:

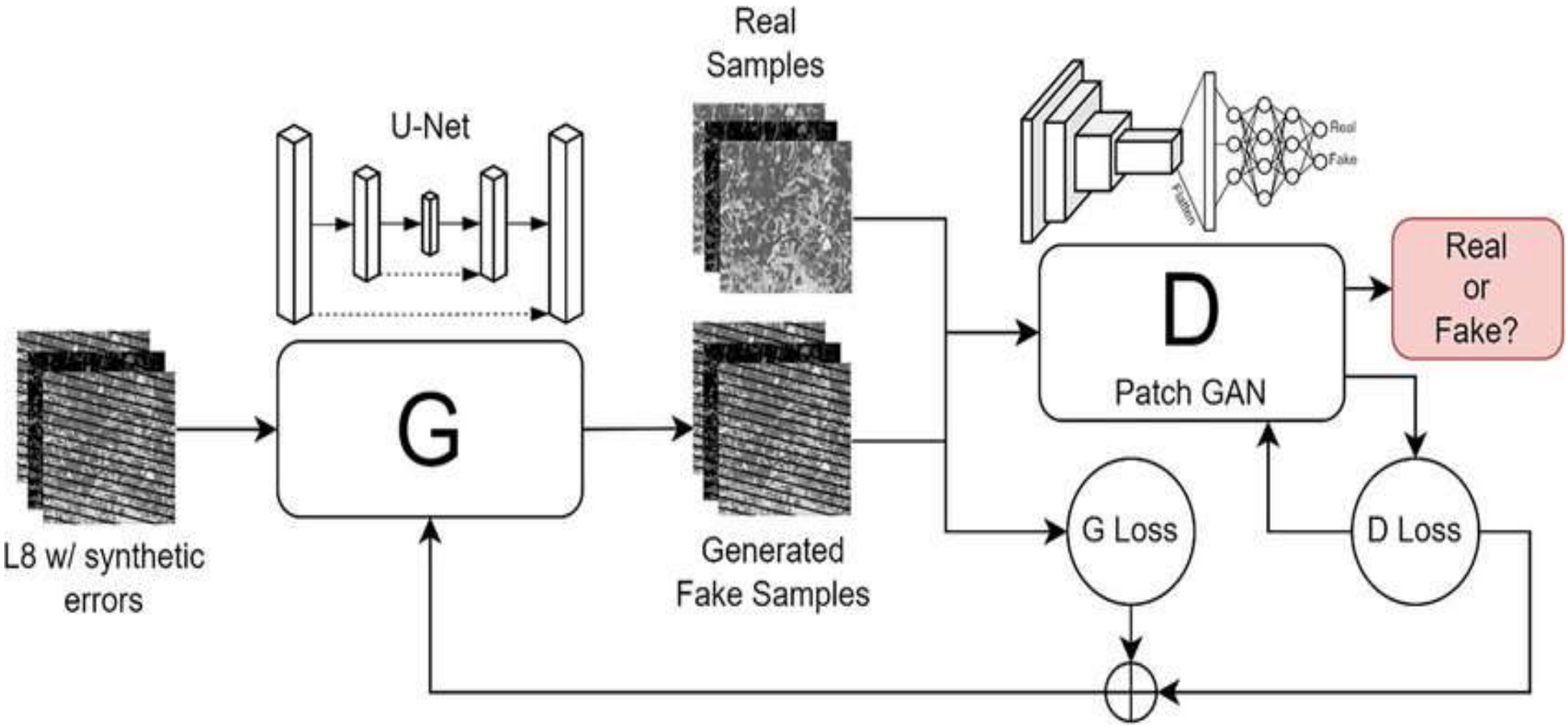
- Image-to-image translation refers to the task of converting an image from one visual representation to another while preserving its structural information.
- In this task, the input and output images are semantically related and usually aligned pixel by pixel.
- The model learns a function that maps each input image to its corresponding target image.
- This process requires paired datasets, where each input image has a matching ground-truth output image.
- Pix2Pix is specifically designed to handle such supervised translation problems effectively.

Image-to-Image Translation:

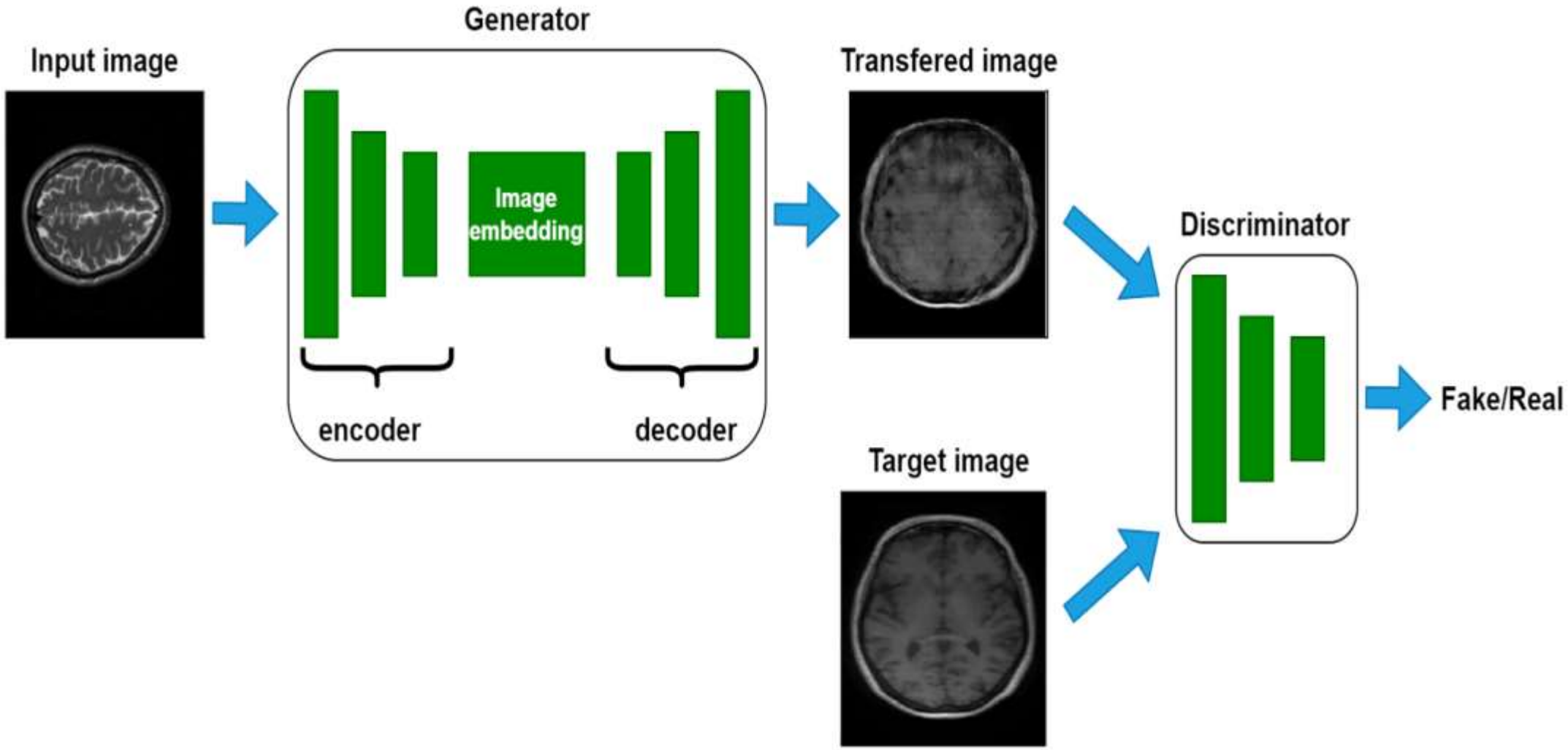
Architecture of Pix2Pix:

- The Pix2Pix architecture consists of two main components: a generator and a discriminator.
- The generator is responsible for generating the target image from the input image.
- The discriminator evaluates whether the generated image is real or fake by comparing it with the actual target image.
- Both networks are trained simultaneously in an adversarial manner.
- The generator tries to fool the discriminator, while the discriminator tries to correctly identify real and generated images.

Architecture of Pix2Pix:



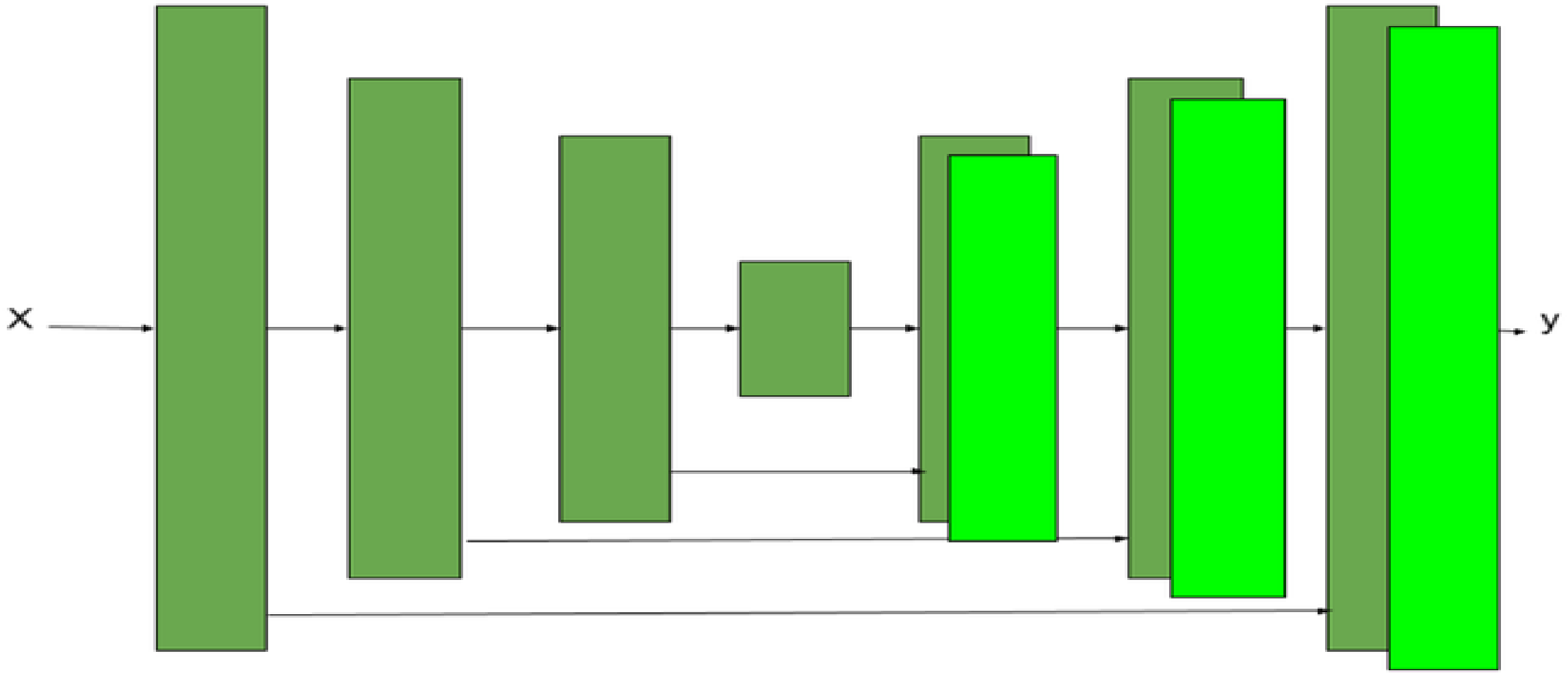
Architecture of Pix2Pix:



Generator Architecture – U-Net:

- The generator in Pix2Pix uses a U-Net architecture, which is an encoder-decoder network with skip connections.
- The encoder compresses the input image into a low-dimensional representation by extracting high-level features.
- The decoder then reconstructs the image from these features to produce the output image.
- Skip connections are added between corresponding encoder and decoder layers.
- These connections help retain spatial and low-level information that would otherwise be lost during downsampling.

U-Net architecture:



Encoder Network in Generator:

- The encoder network in the generator consists of seven convolutional blocks.
- Each block includes a convolution layer followed by a LeakyReLU activation function with a slope of 0.2.
- Batch normalization is applied in all layers except the first one to stabilize training.
- As the image passes through the encoder, its spatial resolution decreases while the number of feature channels increases.
- The encoder focuses on learning high-level semantic features from the input image.

Decoder Network in Generator:

- The decoder network consists of seven transposed convolutional blocks used for upsampling.
- Each block contains an upsampling operation followed by a convolution layer, batch normalization, and a ReLU activation function.
- The decoder gradually increases the spatial resolution of the feature maps.
- Its main purpose is to reconstruct the output image from the encoded representation.
- Skip connections from the encoder provide additional spatial information to improve image quality.

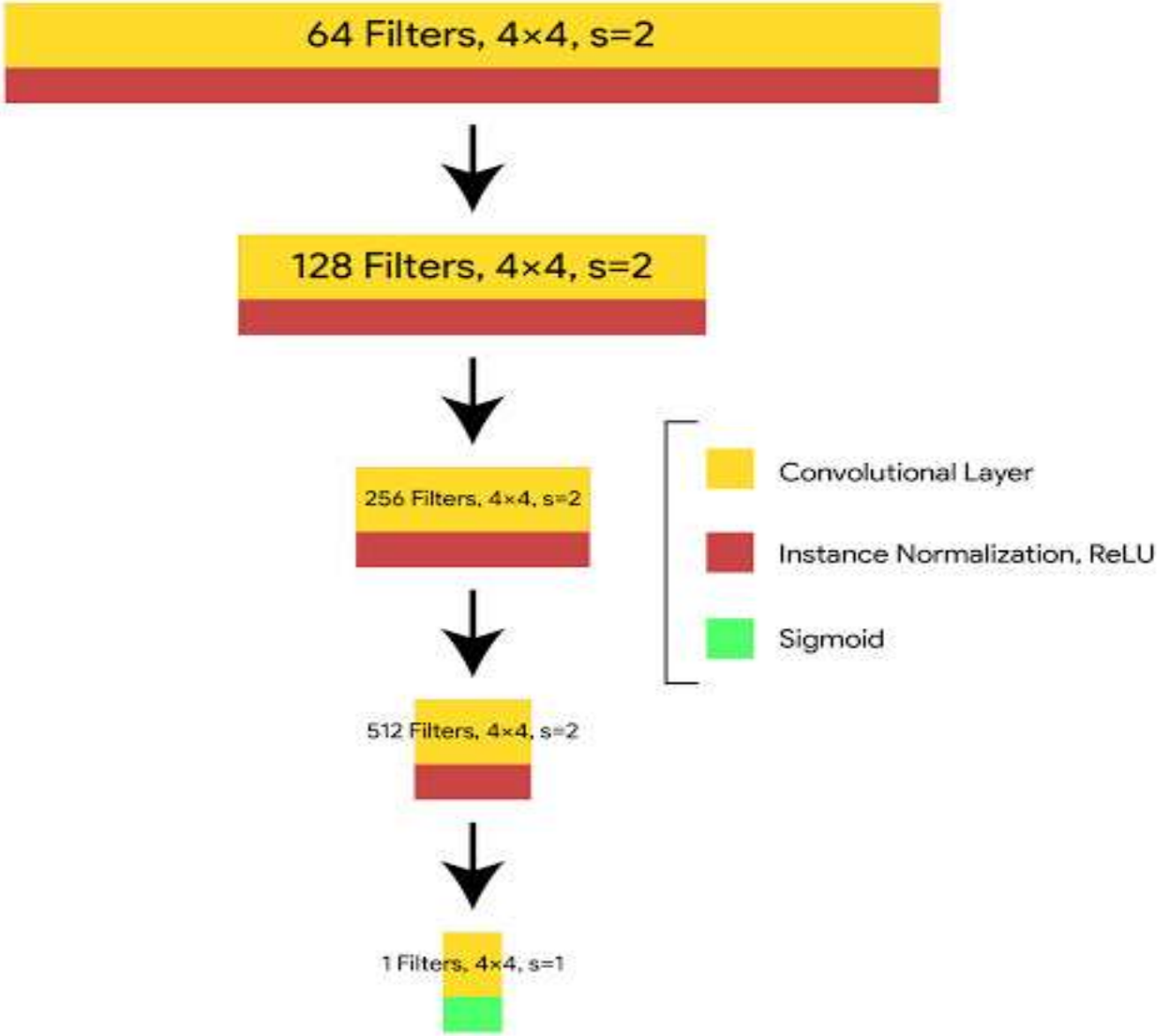
Skip Connections in Pix2Pix:

- Skip connections directly connect encoder layers to corresponding decoder layers.
- These connections concatenate feature maps instead of summing them.
- The encoder captures what features are present in the image, while the decoder needs information about where those features are located.
- Skip connections prevent the loss of fine details such as edges and textures.
- As a result, the generated images are sharper and more accurate.

Discriminator Architecture – PatchGAN:

- Pix2Pix uses a PatchGAN discriminator instead of classifying the entire image at once.
- PatchGAN divides the input image into small fixed-size patches, such as 70×70 pixels, and evaluates each patch independently.
- Each patch is classified as real or fake based on texture and local patterns.
- Instead of producing a single output value, the discriminator produces a grid of real/fake predictions.
- The final discriminator output is obtained by averaging all patch-level predictions.
- This design allows the discriminator to focus on fine details like edges, textures, and sharp transitions in the image.

PatchGAN:


64 Filters, 4×4, s=2
128 Filters, 4×4, s=2
256 Filters, 4×4, s=2
512 Filters, 4×4, s=2
1 Filters, 4×4, s=1
Convolutional Layer
Instance Normalization, ReLU
Sigmoid

Discriminator:

- During training, the discriminator receives pairs of images instead of single images.
- The first input pair contains the original input image and the corresponding real target image.
- This pair represents a correct and realistic mapping and should be classified as real by the discriminator.
- The second input pair contains the same input image along with the image generated by the generator.
- This pair represents an incorrect or fake mapping and should be classified as fake.

Cont..

- The discriminator compares the real and generated image pairs to learn the difference between them.
- By seeing the same input image paired with both real and fake outputs, the discriminator focuses on output quality rather than input content.
- This conditional setup ensures that the discriminator checks whether the generated image correctly corresponds to the input image.
- Over time, the discriminator becomes better at detecting unrealistic textures and errors.
- This forces the generator to improve its outputs to fool the discriminator.

L1 Loss and L2 Loss in Pix2Pix:

- L1 loss and L2 loss are reconstruction loss functions used to measure the difference between the generated image and the target image.
- L1 loss calculates the absolute difference between corresponding pixels, while L2 loss calculates the squared difference between pixels. L2 loss heavily penalizes large errors, which causes the model to average pixel values and results in blurry images.
- L1 loss treats all errors uniformly and preserves edges and fine details better.
- For image-to-image translation tasks, maintaining sharpness and structure is important, so Pix2Pix prefers L1 loss over L2 loss.